

SADHIX

Terraform Quick Notes & CLI Reference

Write Once, Provision Anywhere.

Index

Sr. No	Topic Name	Page No
1.	Terraform CLI Commands	3 – 4
2.	What is Terraform?	5
3.	Core Concepts	6
4.	HCL Syntax & Project Files	7
5.	The Core Workflow	8
6.	Variables & Outputs In Depth	9
7.	State Management	10
8.	Remote Backends	11
9.	Modules Architecture	12
10.	Meta-Arguments, Conditions & Workspaces	13
11.	Providers & a Real Example	14
12.	Best Practices & Pro Tips	15

Chapter 1 Terraform CLI Commands

Cheat sheet for Infrastructure as Code. The everyday loop is **Write, Plan, Apply, Manage, Scale**.

Initialization

Command	Description
terraform init	Initialize working directory
terraform init -upgrade	Re-initialize with latest provider versions
terraform -v	Check Terraform version
terraform providers	Show required providers

Working with Configuration

Command	Description
terraform fmt	Format .tf files
terraform validate	Validate configuration
terraform plan	Preview changes
terraform plan -out=tfplan	Save plan to file
terraform show tfplan	Show saved plan details

Planning and Applying

Command	Description
terraform plan	Show execution plan
terraform apply	Apply changes (ask for confirm)
terraform apply -auto-approve	Apply without prompt
terraform apply tfplan	Apply from saved plan file

Inspect and Debug

Command	Description
terraform show	Show current state or plan
terraform state list	List all resources
terraform state show <addr>	Show specific resource
terraform console	Interactive console
terraform graph	Generate dependency graph

Manage State

Command	Description
terraform state mv <src> <dest>	Move resource
terraform state rm <resource>	Remove from state
terraform state pull	Download state file
terraform state push	Upload state file
terraform state replace-provider	Replace provider in state

Dependency & Lock File

Command	Description
terraform lock	Create .terraform.lock.hcl
terraform lock -platform=linux_amd64	Generate lock for specific platform
terraform lock -upgrade	Update provider checksums
terraform providers lock	Lock providers for all platforms

Modules

Command	Description
terraform get	Download modules
terraform init -upgrade	Upgrade modules/providers
terraform providers lock	Lock module providers

Workspaces

Command	Description
terraform workspace list	List workspaces
terraform workspace new <name>	Create new workspace
terraform workspace select <name>	Switch workspace
terraform workspace delete <name>	Delete workspace

Destroy Infrastructure

Command	Description
terraform destroy	Destroy all resources
terraform destroy -target=<addr>	Destroy specific resource
terraform destroy -auto-approve	Destroy without prompt

Import Existing Resources

Command	Description
terraform import <addr> <id>	Import existing resource
terraform plan	Verify changes after import
terraform apply	Update state with imported resource

Other Useful Commands

Command	Description
terraform version	Show Terraform version
terraform login	Login to Terraform Cloud
terraform logout	Logout from Terraform Cloud
terraform force-unlock <id>	Unlock a locked state
terraform taint <addr>	Mark resource for recreation
terraform untaint <addr>	Remove taint from resource

Chapter 2 What is Terraform?

Terraform is an open-source **Infrastructure as Code (IaC)** tool built by HashiCorp. Instead of clicking through a cloud console to create servers, networks or databases, you describe the infrastructure in text files and Terraform builds it consistently, every time.

Terraform is **declarative**: you describe the desired end state (for example "two web servers and one load balancer") and Terraform works out the API calls needed to reach it, whether starting from nothing or adjusting existing infrastructure.

Configuration is written in **HCL** (HashiCorp Configuration Language) in files with a `.tf` extension.

```
# main.tf
resource "aws_instance" "web" {
  ami           = "ami-0123456789abcdef0"
  instance_type = "t2.micro"
  tags = {
    Name = "web-server"
  }
}
```

Running `terraform apply` reads this file, compares it with what already exists, and creates exactly one EC2 instance matching the description.

Core Benefits

- **Automation** - provision entire environments with one command.
- **Consistency** - same code gives the same infrastructure in dev, staging and production.
- **Scalability** - go from one server to a thousand by changing a number.
- **Version Control** - keep infrastructure in Git, review in pull requests, roll back when needed.

Chapter 3 Core Concepts

Every configuration is built from six building blocks: providers, resources, data sources, modules, state, and variables/outputs.

Providers

A provider is a plugin that lets Terraform talk to a platform API (AWS, Azure, GCP, Kubernetes, GitHub and more). It is downloaded during `terraform init`.

```
provider "aws" {  
  region = "us-east-1"  
}
```

Resources

A resource is the actual infrastructure object Terraform manages. It is identified by a **type** and a local **name**.

```
resource "aws_s3_bucket" "app_logs" {  
  bucket = "my-app-logs-2026"  
}
```

Data Sources

A data source is a read-only lookup of something that already exists (reference:

`data.aws_ami.latest_ubuntu.id`).

```
data "aws_ami" "latest_ubuntu" {  
  most_recent = true  
  owners      = ["099720109477"]  
  filter {  
    name   = "name"  
    values = ["ubuntu/images/*22.04*"]  
  }  
}
```

Modules

A module is a reusable group of resources; every configuration is itself the root module.

```
module "vpc" {  
  source      = "../modules/vpc"  
  cidr_block = "10.0.0.0/16"  
}
```

State

State maps every resource in code to the real object in the cloud (ID and attributes). It is stored in `terraform.tfstate`.

Variables and Outputs

Variables make config dynamic (`var.instance_type`); outputs expose values such as an IP.

```
variable "instance_type" {  
  type      = string  
  default   = "t2.micro"  
}  
output "instance_ip" {  
  value = aws_instance.web.public_ip  
}
```

Chapter 4 HCL Syntax & Project Files

Every block has a keyword, optional quoted labels, and a body in curly braces holding arguments. Terraform loads every `.tf` file in a directory as one configuration; splitting files by purpose is only a convention.

```
<block_type> "<label>" "<label>" {  
  argument = value  
}
```

File	Purpose
main.tf	Resources, data sources and module calls (entry point)
variables.tf	variable blocks: type, description, default
outputs.tf	output blocks that print or export results
versions.tf	Required Terraform and provider versions
terraform.tfvars	Actual values for declared variables (often one per environment)

```
terraform {  
  required_version = ">= 1.7.0"  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```

Created automatically

- `.terraform/` - downloaded provider plugins. Safe to delete; recreated by `init`.
- `.terraform.lock.hcl` - exact provider versions used. **Commit this file to Git.**

Chapter 5 The Core Workflow

Almost every session follows the same loop. Skipping `plan` is a common cause of unwanted production changes.

Command	Description
<code>terraform init</code>	Download providers/modules and configure the backend. Run once, and again after adding a provider or module
<code>terraform plan</code>	Compare config with state and show what will be created, changed or destroyed. Touches nothing
<code>terraform apply</code>	Execute the plan and make real API calls. Asks for confirmation unless <code>-auto-approve</code>
<code>terraform destroy</code>	Remove every resource this configuration manages. Dangerous in production

Supporting commands

- `terraform fmt` - rewrite files into the standard style.
- `terraform validate` - check syntax and consistency without contacting the cloud.
- `terraform plan -out=tfplan` - save a plan so apply runs exactly what was reviewed.
- `terraform output` - reprint output values without re-applying.

Remember: Always read the plan before you apply. Reserve `-auto-approve` for CI/CD pipelines.

Chapter 6 Variables & Outputs In Depth

Types & validation

Besides string and number, Terraform supports list, map, bool and object types. A `validation` block rejects bad input before anything is created.

```
variable "environment" {
  type          = string
  description   = "Deployment environment"
  validation {
    condition    = contains(["dev","staging","prod"], var.environment)
    error_message = "environment must be dev, staging or prod."
  }
}
```

Sensitive variables

Marking a variable `sensitive` hides its value from CLI output and logs.

```
variable "db_password" {
  type          = string
  sensitive     = true
}
```

Supplying values

Values come from `terraform.tfvars`, command-line flags, or environment variables prefixed with `TF_VAR_` (good for secrets in CI/CD).

```
# terraform.tfvars
environment = "prod"

# or from the shell
export TF_VAR_db_password="s3cr3t"
```

Structured outputs

Outputs can return lists or maps, useful with `count` and `for_each`.

```
output "instance_ips" {
  value = { for k, v in aws_instance.web : k => v.public_ip }
}
```

Chapter 7 State Management

`terraform.tfstate` is a JSON file mapping each resource address to the real object and its attributes. Every plan and apply reads it.

Resource in code	Real-world object (ID)
<code>aws_instance.web</code>	<code>i-0abc123def456789</code>
<code>aws_s3_bucket.logs</code>	<code>my-app-logs-2026</code>
<code>module.vpc.aws_vpc.main</code>	<code>vpc-0a1b2c3d4e5f</code>

Drift & refresh

If someone changes a resource outside Terraform, state falls out of sync. This is **drift**. `terraform apply -refresh-only` re-syncs state without changing infrastructure.

State locking

Terraform locks state during writes so two runs cannot corrupt it. If a crashed run leaves a stale lock, release it with `terraform force-unlock LOCK_ID`, only when sure no other run is active.

Useful state commands

Command	Description
<code>terraform state list</code>	List every tracked resource
<code>terraform state show <addr></code>	Inspect one resource
<code>terraform state mv <src> <dest></code>	Move / rename a resource in state
<code>terraform state rm <addr></code>	Stop tracking (does NOT delete it)
<code>terraform state pull</code>	Download state file
<code>terraform state push</code>	Upload state file
<code>terraform import <addr> <id></code>	Bring an existing resource under management

Golden rule: Never edit `terraform.tfstate` by hand and never commit it to Git; it can contain secrets in plain text.

Chapter 8 Remote Backends

A local state file breaks down with more than one person: files can disagree and there is no locking. A remote backend stores state in a shared, versioned, lockable place.

Backend	Where state lives	Locking
AWS S3 + DynamoDB	S3 bucket (versioning + encryption)	DynamoDB table holds the lock
Terraform Cloud	Managed by HashiCorp	Built in, plus remote runs
Azure Storage	Blob container	Native blob lease
Google Cloud Storage	GCS bucket	Native locking

```
terraform {
  backend "s3" {
    bucket      = "my-terraform-state"
    key         = "prod/network/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt     = true
  }
}
```

After adding or changing a backend block, run `terraform init` again; Terraform offers to migrate existing state automatically.

A remote backend gives you

- Shared visibility for every teammate and CI job.
- Locking that prevents concurrent applies.
- Encryption at rest and versioned history for recovery.

Chapter 9 Modules Architecture

A module defines a pattern (VPC, subnets, security groups) once and reuses it with different inputs, like a function in a program.

Directory layout

```
main.tf          # calls the module
variables.tf
outputs.tf
modules/
  vpc/
    main.tf
    variables.tf
    outputs.tf
```

Local module

```
module "vpc" {
  source      = "../modules/vpc"
  cidr_block = "10.0.0.0/16"
}

output "vpc_id" {
  value = module.vpc.vpc_id
}
```

Registry module

Always pin a version so an update cannot change your infrastructure unexpectedly.

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"
  name    = "app-vpc"
  cidr    = "10.0.0.0/16"
}
```

Benefits

- **Reusability** - write the network once, use it everywhere.
- **Maintainability** - fix a bug in one place.
- **DRY** - no copy-pasted resource blocks.

Chapter 10 Meta-Arguments, Conditions & Workspaces

count & for_each

`count` creates a fixed number of identical copies (index-based, read all with the `[*]` splat). `for_each` creates one resource per map or set entry, ideal when each copy differs. Inside the block use `each.key` and `each.value`; addresses look like `aws_instance.web["a"]`.

```
resource "aws_instance" "web" {
  for_each      = toset(["a", "b"])
  instance_type = "t2.micro"
  tags = { Name = "web-${each.key}" }
}
```

depends_on & conditionals

`depends_on` forces an explicit build order. The ternary condition `? true_value : false_value` picks a value inline.

```
resource "aws_instance" "web" {
  instance_type = var.environment == "prod" ? "t2.medium" : "t2.micro"
  depends_on    = [aws_security_group.web_sg]
}
```

Workspaces

A workspace gives the same configuration a separate state file per environment, like a Git branch. Many teams prefer fully separate state or directories for production.

Command	Description
<code>terraform workspace list</code>	List workspaces
<code>terraform workspace new <name></code>	Create new workspace
<code>terraform workspace select <name></code>	Switch workspace
<code>terraform workspace delete <name></code>	Delete workspace

Choosing: `count` for simple identical repeats, `for_each` when each copy needs its own name or setting, workspaces (or separate state) for independent environments.

Chapter 11 Providers & a Real Example

Source address	Manages
hashicorp/aws	EC2, S3, VPC, IAM, RDS, EKS and more
hashicorp/azurerm	Virtual machines, storage accounts, AKS
hashicorp/google	Compute Engine, Cloud Storage, GKE
hashicorp/kubernetes	Deployments, services, config maps
kreuzwerker/docker	Images, containers, networks, volumes

A minimal real-world example

A key pair, a security group, and an EC2 instance using variables and printing its public IP.

```
resource "aws_key_pair" "deployer" {
  key_name   = "deployer-key"
  public_key = file("deployer.pub")
}

resource "aws_security_group" "web_sg" {
  name = "web-sg"
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  key_name          = aws_key_pair.deployer.key_name
  security_groups   = [aws_security_group.web_sg.name]
  instance_type     = var.instance_type
  ami               = var.ami_id
}

output "public_ip" {
  value = aws_instance.web.public_ip
}
```

Note: Opening SSH to 0.0.0.0/0 is fine for learning only. Restrict the CIDR range in real environments.

Chapter 12 Best Practices & Pro Tips

- Store state remotely, with locking and encryption turned on.
- Pin Terraform and provider versions in `versions.tf`.
- Always run `terraform init` before plan or apply.
- Run `terraform fmt` and `terraform validate` before every commit (and in CI/CD).
- Always read the plan output before running apply.
- Use workspaces for dev, staging and production, or separate state for stronger isolation.
- Keep `.terraform.lock.hcl` under version control.
- Use `-auto-approve` only in automation, not local dev.
- Keep secrets out of code and Git; use a secrets manager.
- Build small, focused modules with consistent naming and tags.
- Apply changes through CI/CD rather than a personal laptop.

Pro Tip: Treat Terraform code exactly like application code: Git, pull requests, peer review, automated plan in CI, tagged releases.

Plan - Apply - Scale - Repeat